



**UNIVERSIDAD
DON BOSCO**

DSP404 — Software Propietario



Semana 8

Introducción al Desarrollo Web

ASP.NET Core · Razor Pages · Formularios · Validación

Cliente vs Servidor

Razor Pages

Formularios

Validación

¿Qué lograrás al terminar esta clase?

01

Explicas la diferencia cliente vs servidor

Qué corre en el navegador y qué corre en el servidor.

02

Distingues web estática de web dinámica

HTML fijo vs páginas generadas por código en tiempo real.

03

Creas tu primer proyecto ASP.NET Core Razor Pages

Estructura de carpetas, Program.cs, el modelo de páginas.

04

Construyes un formulario con validación

Data Annotations y validación del lado del servidor.

05

Manejas errores y excepciones en web

try/catch y páginas de error amigables.

01

La Web: Cliente y Servidor

Cómo funciona internet por dentro

02

Estático vs Dinámico

HTML fijo vs generado por código

03

ASP.NET Core y Razor Pages

El framework y su estructura

04

Formularios y Validación

Capturar y validar datos del usuario

05

Manejo de Errores

try/catch y páginas de error

01

La Web

Cliente y Servidor — cómo funciona internet

¿Qué pasa cuando abres una página web?

💡 Toda la web funciona con un modelo PETICIÓN → RESPUESTA. Tu navegador (cliente) PIDE algo. Un servidor lo PROCESA y RESPONDE. Entender esto es la base de todo el desarrollo web.



🔑 Cuando escribes una URL y presionas Enter, tu navegador manda una PETICIÓN al servidor. El servidor responde con HTML que el navegador dibuja en pantalla.

¿Qué corre en el cliente y qué en el servidor?



En el CLIENTE (navegador)

- ✓ HTML — estructura de la página
- ✓ CSS — estilos y diseño
- ✓ JavaScript — interactividad
- ✓ Lo que el usuario VE y toca
- ✓ Ya lo conocen de sus clases previas



En el SERVIDOR (ASP.NET)

- ✓ C# — lógica del programa
- ✓ Acceso a base de datos
- ✓ Validación de seguridad
- ✓ Generación del HTML dinámico
- ✓ Lo que aprenden a partir de hoy



Regla de oro de seguridad: NUNCA confíes solo en la validación del cliente (JavaScript). El usuario puede saltársela. La

HTTP — el idioma entre cliente y servidor

💡 HTTP (HyperText Transfer Protocol) es el conjunto de reglas con que el cliente y el servidor se comunican. Cada petición usa un VERBO que dice qué quieres hacer.

GET**Pedir / leer datos**

Ver una página, cargar una lista de productos

POST**Enviar / crear datos**

Enviar un formulario, registrar un usuario

PUT**Actualizar datos**

Editar un producto existente

DELETE**Borrar datos**

Eliminar un registro

📌 Por ahora nos enfocamos en GET (mostrar páginas) y POST (enviar formularios). PUT y DELETE los verán al construir

Estático vs Dinámico

HTML fijo vs páginas generadas por código

Web estática vs Web dinámica

Web ESTÁTICA

El HTML es fijo. Todos los usuarios ven exactamente lo mismo. El archivo .html ya existe en el servidor tal cual.

- Páginas .html escritas a mano
- El contenido NO cambia
- Mismo resultado para todos
- Ej: una página 'Acerca de'

```
<!-- index.html -->
<h1>Bienvenido</h1>
<p>Somos una tienda.</p>
```

Web DINÁMICA

El HTML se GENERA en el momento, con código C#. Cada usuario puede ver contenido distinto según sus datos.

- Páginas generadas por código
- El contenido cambia según datos
- Personalizado por usuario
- Ej: 'Hola, Ana' con TUS pedidos

```
<!-- Razor: .cshtml -->
<h1>Hola, @Model.Nombre</h1>
<p>Tienes @Model.Pedidos pedidos.</p>
```

03 ASP.NET Core

El framework y Razor Pages

¿Qué es ASP.NET Core? — el concepto

💡 ASP.NET Core es el framework de Microsoft para construir aplicaciones web con C#. Es gratuito, multiplataforma y muy rápido. Razor Pages es la forma más simple de empezar.



Gratuito y open source

Sin licencias. Corre en Windows, Linux y macOS.



Muy rápido

De los frameworks web más veloces del mercado.



Razor Pages

Modelo simple: cada página tiene su HTML y su código C#.



MVC (más adelante)

Para apps grandes. Lo verán en la semana 13.



Web APIs

Para servicios y apps móviles. Semana 15.



Integra bases de datos

Con Entity Framework Core. Semana 14.

Estructura de un proyecto Razor Pages

MiPrimeraWebApp/

Pages/

Index.cshtml

Index.cshtml.cs

Privacy.cshtml

Shared/

_Layout.cshtml

wwwroot/

css/ js/ img/

Program.cs

appsettings.json

Pages/

Aquí van todas tus páginas web

.cshtml

El HTML + Razor (lo que el usuario ve)

.cshtml.cs

El código C# de esa página (PageModel)

wwwroot/

Archivos estáticos: CSS, JS, imágenes

Program.cs

Configura y arranca la aplicación

_Layout.cshtml

Plantilla común para todas las páginas

El par de archivos: Vista + PageModel

💡 Cada página Razor son DOS archivos que trabajan juntos: el .cshtml (la vista — qué se ve) y el .cshtml.cs (el PageModel — la lógica en C#).

Index.cshtml (la Vista)

```
@page
@model IndexModel

<h1>@Model.Saludo</h1>

<p>La hora actual es:
    @Model.HoraActual</p>

@foreach (var p in Model.Productos)
{
    <div>@p</div>
}
```

Index.cshtml.cs (el PageModel)

```
public class IndexModel : PageModel
{
    public string Saludo { get; set; }
    public string HoraActual { get; set; }
    public List<string> Productos { get; set; }
}

public void OnGet() // se ejecuta al
cargar
{
    Saludo = "Bienvenido a la tienda";
    HoraActual =
        DateTime.Now.ToString("HH:mm");
    Productos = new() { "Lantón", "Mouse"
}
```

🔑 El @model conecta los dos archivos. @Model.Saludo en la vista muestra el valor que el PageModel asignó en OnGet().

Program.cs — el corazón de la aplicación

```
// Program.cs — punto de entrada de la app web (.NET 10)
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// 1. Registrar servicios: habilitar Razor Pages  
builder.Services.AddRazorPages();
```

```
var app = builder.Build();
```

```
// 2. Configurar el pipeline (cómo se procesan las  
peticiones)
```

```
if (!app.Environment.IsDevelopment())  
{
```

```
    app.UseExceptionHandler("/Error"); // página de  
    error en producción  
}
```

```
app.UseStaticFiles(); // servir CSS, JS, imágenes
```

¿Qué hace cada parte?

1

CreateBuilder

Prepara la configuración de la app

2

AddRazorPages

Activa el sistema de páginas Razor

3

UseStaticFiles

Permite servir CSS/JS/imágenes

4

MapRazorPages

Conecta las URLs con las páginas

5

app.Run()

Arranca el servidor web

04

Formularios

Capturar y validar datos del usuario

¿Cómo viajan los datos de un formulario?

💡 Cuando el usuario llena un formulario y presiona 'Enviar', el navegador hace una petición POST con los datos. El método `OnPost()` en el `PageModel` los recibe.

1

Usuario llena
el formulario

2

Presiona
'Enviar' (POST)

3

`OnPost()` recibe
los datos

4

Se validan
los datos

5

Respuesta al
usuario

📌 `OnGet()` se ejecuta cuando se CARGA la página (mostrar el formulario vacío). `OnPost()` se ejecuta cuando se ENVÍA (procesar los datos). Dos momentos distintos.

Formulario con Razor — vista y modelo

Registro.cshtml (formulario)

```
@page
@model RegistroModel

<form method="post">
  <label>Nombre:</label>
  <input asp-for="Nombre" />
  <span asp-validation-for="Nombre"></span>

  <label>Email:</label>
  <input asp-for="Email" />
  <span asp-validation-for="Email"></span>

  <label>Edad:</label>
  <input asp-for="Edad" />
  <span asp-validation-for="Edad"></span>

  <button type="submit">Registrar</button>
</form>
```

```
@if (Model.Exito)
```

Registro.cshtml.cs (PageModel)

```
public class RegistroModel : PageModel
{
    [BindProperty]
    public string Nombre { get; set; }

    [BindProperty]
    public string Email { get; set; }

    [BindProperty]
    public int Edad { get; set; }

    public bool Exito { get; set; }

    public void OnGet() { } // muestra el form

    public IActionResult OnPost()
    {
        if (!ModelState.IsValid)
            return Page(); // hay errores,
    }
}
```

```
recargar
```



[BindProperty] conecta el campo del formulario con la propiedad C#. asp-for genera el HTML correcto

Data Annotations — validar sin escribir if

💡 Las Data Annotations son etiquetas [entre corchetes] que pones sobre una propiedad para definir sus reglas de validación. ASP.NET las revisa automáticamente.

```
public class RegistroModel : PageModel
{
    [BindProperty]
    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(50, MinimumLength = 3)]
    public string Nombre { get; set; }

    [BindProperty]
    [Required]
    [EmailAddress(ErrorMessage = "Email inválido")]
    public string Email { get; set; }

    [BindProperty]
    [Range(18, 120, ErrorMessage = "Edad entre 18 y
120")]
    public int Edad { get; set; }
```

Anotaciones comunes

[Required]	Campo obligatorio
[StringLength]	Largo máx/mín del texto
[EmailAddress]	Formato de email válido
[Range (a,b)]	Número en un rango
[Phone]	Formato de teléfono
[Compare]	Comparar 2 campos (contraseñas)

💡 Ventaja enorme: en lugar de escribir 20 líneas de if para validar, pones una etiqueta. ASP.NET genera los mensajes de

05

Errores

Manejo de excepciones y páginas de error

¿Qué es una excepción? — try / catch

💡 Una excepción es un error que ocurre mientras el programa corre (dividir por cero, archivo no encontrado, conexión perdida). Si no se maneja, el programa se cae. try/catch lo atrapa.

```
try
{
    // Código que PODRÍA fallar
    int resultado = 100 / divisor;
    var datos = LeerArchivo("config.txt");
    GuardarEnBaseDeDatos(datos);
}
catch (DivideByZeroException ex)
{
    // Maneja un error específico
    Console.WriteLine("No se puede dividir por cero");
}
catch (Exception ex)
{
    // Maneja cualquier otro error
}
```

Anatomía del try/catch

try

El código que podría fallar

catch

Qué hacer si hay error

finally

Siempre se ejecuta al final

ex.Message

El mensaje del error



Atrapa primero las excepciones específicas (DivideByZeroException) y al

Páginas de error amigables en la web

💡 En una web real, NUNCA muestres el error técnico al usuario (revela información y se ve mal). Muestra una página de error amigable y registra el detalle en un log para ti.

❌ Lo que NO debes mostrar

```
System.NullReferenceException:  
Object reference not set  
    at MyApp.Productos.OnGet()  
    in C:\Users\dev\Productos.cs:line 42  
    at System.RuntimeMethodHandle...  
[Stack trace completo visible]
```

✅ Lo que SÍ debes mostrar



¡Ups! Algo salió mal

Estamos trabajando en ello.
Intenta de nuevo en unos minutos.

```
// En Program.cs: configurar la página de error  
if (!app.Environment.IsDevelopment())
```



Demo en vivo — Formulario de Registro

El docente crea un proyecto Razor Pages desde cero
y construye un formulario de registro con validación

Pasos del ejercicio explicado en vivo

- 1 Crear el proyecto**
`dotnet new webapp -n MiWebApp` (o VS → ASP.NET Core Web App)
- 2 Explorar la estructura**
Pages/, wwwroot/, Program.cs — mostrar cada parte
- 3 Crear página Registro**
Click derecho en Pages → Agregar → Razor Page → 'Registro'
- 4 Escribir el formulario**
En Registro.cshtml: `<form method='post'>` con asp-for
- 5 Agregar el PageModel**
En Registro.cshtml.cs: propiedades `[BindProperty]` + `OnPost()`
- 6 Añadir validación**
Data Annotations: `[Required]`, `[EmailAddress]`, `[Range]`
- 7 Ejecutar y probar**
F5 → llenar el form con datos válidos e inválidos



¡Tu turno! — Formulario de Contacto

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide



Crea una página Razor 'Contacto' con un formulario que capture y valide estos datos:

- | | |
|--------------------------|--|
| • Nombre completo | Obligatorio, entre 3 y 60 caracteres |
| • Email | Obligatorio, formato de email válido |
| • Teléfono | Formato de teléfono ([Phone]) |
| • Asunto | Obligatorio, máximo 100 caracteres |
| • Mensaje | Obligatorio, entre 10 y 500 caracteres |
| • Acepta términos | Debe estar marcado (checkbox, bool) |

Comportamiento requerido:

- Si hay errores → mostrar los mensajes de validación y NO procesar.
- Si todo es válido → mostrar un mensaje de éxito '  Mensaje enviado, te contactaremos pronto!'

❌ Olvidar [BindProperty] — los datos llegan vacíos

```
public string Nombre { get; set; }  
// ❌ sin [BindProperty]  
// OnPost recibe Nombre = null
```

```
[BindProperty] // ✅  
public string Nombre { get; set; }
```

❌ No verificar ModelState.IsValid antes de procesar

```
public IActionResult OnPost() {  
    Guardar(); // ❌ guarda aunque haya errores  
    return Page();  
}
```

```
public IActionResult OnPost() {  
    if (!ModelState.IsValid) return Page(); // ✅  
    Guardar();  
    return Page();  
}
```

❌ Olvidar method='post' en el formulario

```
<form> <!-- ❌ por defecto es GET -->  
    <input asp-for="Nombre" />  
</form>
```

```
<form method="post"> <!-- ✅ -->  
    <input asp-for="Nombre" />  
</form>
```

Semana 8 — Introducción al Desarrollo Web

- ✓ **Cliente vs Servidor —** El navegador pide (HTML/CSS/JS), el servidor responde (C#).
- ✓ **HTTP —** GET = pedir/mostrar, POST = enviar/crear datos.
- ✓ **Estático vs Dinámico —** HTML fijo vs HTML generado por código con @Model.
- ✓ **Razor Pages —** Cada página = .cshtml (vista) + .cshtml.cs (PageModel).
- ✓ **OnGet / OnPost —** OnGet muestra la página, OnPost procesa el formulario.
- ✓ **[BindProperty] —** Conecta los campos del formulario con propiedades C#.
- ✓ **Data Annotations —** [Required], [EmailAddress], [Range] validan automáticamente.
- ✓ **try / catch —** Atrapa excepciones. Página de error amigable en producción.



Preparación — Proyecto Fase I

Se entrega en la Semana 9 (10%)

- Aplicación web Razor Pages con al menos 2 formularios validados
- Uso de Data Annotations en todos los campos
- Manejo de errores con try/catch y página de error
- Practica la guía de ejercicios de esta semana

La guía de ejercicios se entrega aparte (versión docente y alumno).